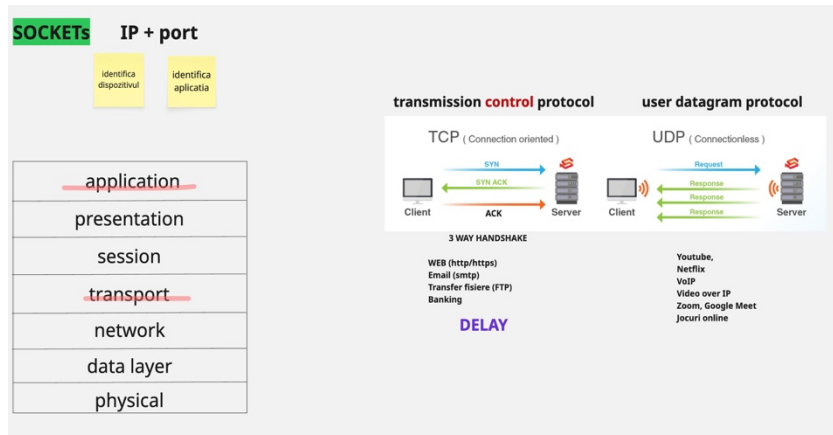


Guideline

Programarea rețelelor cu sockets și interoperabilitate



Exerciții practice: Provocarea Polyglot

Pentru acest exercițiu, formați echipe de câte doi. Trebuie să folosiți **limbaje de programare diferite** (ex: Studentul A folosește Python, Studentul B folosește Java sau C#).

Misiunea A: Chat Fiabil (TCP)

Creați o aplicație de chat tip "Ping-Pong" (trimitere și primire secvențială).

- **Flux:** Serverul acceptă conexiunea și așteaptă. Clientul trimite primul mesaj. Serverul îl citește, îl afișează, apoi permite tastarea și trimiterea unui răspuns.
- **Detalii tehnice:** Folosiți codificarea **UTF-8**. Bucla trebuie să se oprească grațios (închizând socket-ul) când unul dintre utilizatori trimite mesajul "exit".

Misiunea B: Chat Rapid (UDP)

Rescrieți chat-ul folosind protocolul UDP.

- **Diferența:** Deoarece UDP nu stabilește o conexiune, Serverul va folosi direct funcția de recepție (ex: **recvfrom**). Acesta trebuie să extragă adresa IP și portul din primul pachet primit de la Client pentru a ști unde să trimită răspunsurile.

Analiza traficului cu Wireshark

Analizați pachetele capturate pe interfața Tailscale și completați următoarele observații:

Caracteristică	Protocol TCP	Protocol UDP
Denumire PDU	Segment	Datagramă
Handshake inițial	Prezent (SYN, SYN-ACK, ACK)	Absent
Confirmarea primirii	Pachete de tip ACK	Nu se confirmă primirea

Predarea proiectului prin GitHub Classroom

Fluxul de lucru colaborativ (GitHub Classroom)

Amândoi membrii echipei vor lucra pe **același repository** generat de GitHub Classroom. Sincronizarea corectă este vitală pentru integritatea proiectului.

Intrați pe link-ul corespunzător grupei voastre și acceptați invitația spre GitHub Classroom: <https://moodle.cs.utcluj.ro/mod/page/view.php?id=67655>

Pasul 1: Clonarea. Ambii studenți descarcă repository-ul local:

```
Shell
git clone
https://github.com/nume-org/lab-sockets-echipa-voastra.git
```

Pasul 2: Încărcarea codului (Studentul A). Studentul A dezvoltă codul (într-un folder separat, ex: **Server_Java**), îl salvează și îl publică:

```
Shell
git add .
git commit -m "Implementat server TCP"
git push origin main
```

Pasul 3: Sincronizarea (Studentul B). Înainte ca Studentul B să își încarce propriul cod, acesta **trebuie obligatoriu să descarce munca colegului**. În caz contrar, GitHub va respinge operațiunea de **push**.

```
Shell
git pull origin main
# Acum Studentul B are și folderul Server_Java local.
git add .
git commit -m "Implementat client Python"
git push origin main
```

În fișierul README.md al repository-ului, adăugați:

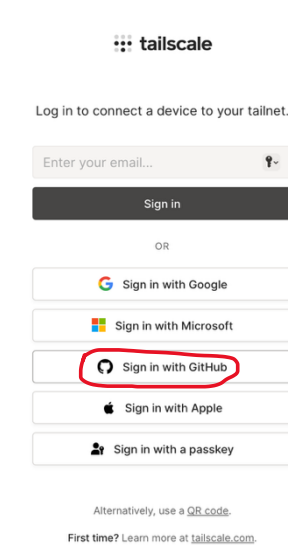
1. O explicație clară a modului de funcționare al aplicației client-server pentru:
 - o TCP (cu conexiune)
 - o UDP (fără conexiune)

Explicația trebuie să includă:

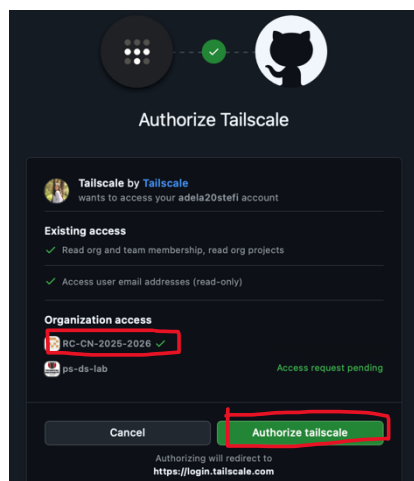
- o cum se realizează conexiunea (pentru TCP)
 - o cum sunt trimise mesajele
 - o diferențele observate între TCP și UDP
2. Două capturi de ecran din Wireshark:
 - o una pentru **TCP** (ex: tcp.port == 5000), în care să fie vizibil handshake-ul și schimbul de date
 - o una pentru **UDP** (ex: udp.port == 5001), în care să fie vizibile pachetele transmise fără handshake

Tutorial: How to connect to Tailscale

- Download Tailscale
- Log in using your Github account



- Authorize Tailscale and allow access to your GitHub account and organization



tailscale adela20stefi@github



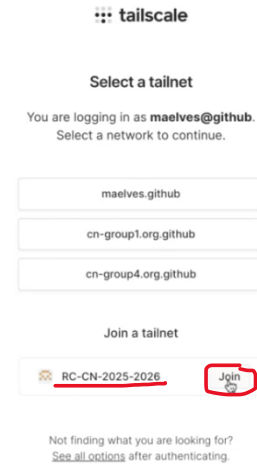
Login successful

Your device Adela is logged in to the rc-cn-2025-2026.org.github tailnet.

If this is not what you meant to do, you can remove the device from your tailnet. If you need help, [contact support](#).

You will be redirected to your console shortly.
Or, you can [visit the console](#) immediately.

- Join the lab Tailscale network



- Wait for approval from the network owner



- You are now connected! You can communicate with your colleagues.

